

Tarea 2 – Integración y composición de componentes personalizados (Swing)

Objetivo

Diseñar y desarrollar un **componente compuesto reutilizable** basado en `JPanel` que **integre varios componentes personalizados** creados en la **Tarea 1 por otros compañeros**, junto con componentes Swing estándar.

El objetivo es practicar:

- La **composición de componentes** en Swing.
- Integrar tu componente personalizado y entregado en la **Tarea 1**.
- La **reutilización de componentes personalizados de terceros**.
- La gestión de propiedades, eventos y diseño de interfaces complejas.
- La documentación técnica del componente creado.

Descripción general

A cada alumno se le asignará un conjunto de **componentes personalizados** desarrollados por sus compañeros en la **Tarea 1**.

Con ellos, junto con su propio componente personalizado (Tarea1) deberá crear un nuevo componente de mayor nivel que:

- Herede de `JPanel`.
- Combine e integre funcionalmente dichos componentes.
- Ofrezca una interacción clara y útil.
- Sea configurable mediante propiedades públicas.
- Pueda reutilizarse sin modificar su código interno desde el plugin de Windowsbuilder.

Requisitos técnicos mínimos

1) Componente base

- Crear una clase, por ejemplo: `public class MiPanelCompuesto extends JPanel`

2) Integración de componentes personalizados

- Integrar **los componentes personalizados** creados por otros alumnos en la Tarea 1 y asignados por el docente.
- Se permite añadir otros componentes Swing estándar (`JLabel`, `JButton`, `JTextField`, etc.).
- Los componentes deben interactuar entre sí de forma lógica (no solo estar colocados).

3) Propiedades propias

- Definir **al menos 2 propiedades nuevas** del panel compuesto.
- Cada propiedad debe:
 - Tener su getter/setter correspondiente.
 - Influir realmente en el comportamiento o apariencia del componente.

4) Eventos e interacción

- El componente debe:
 - Escuchar eventos de los componentes integrados (listeners).
 - Reaccionar ante ellos (actualizar estado, habilitar/deshabilitar elementos, mostrar información, etc.).
- Opcionalmente, el componente compuesto puede:
 - Definir **eventos propios** (listeners personalizados) para notificar acciones externas.

5) Diseño y reutilización

- Usar un **LayoutManager** adecuado (**BorderLayout**, **GridLayout**, etc.).
- El componente debe poder reutilizarse en cualquier proyecto Swing:
 - Sin modificar su código.
 - Configurable sólo mediante propiedades y eventos.

6) Ventana de demostración

- Crear una clase interfaz de demo (**JFrame** o **JDialog**) que:
 - Muestre el funcionamiento del componente compuesto creado.
 - Permitiendo probar sus propiedades y la interacción entre componentes.
- La demo debe (al menos) usar cada uno de los componentes “custom” en base a la documentación de los mismos.

Documentación obligatoria

1. Entregar un documento (**PDF**) que incluya:
 - a. **Descripción general**
 - i. Nombre del componente.
 - ii. Propósito y problema que resuelve.
 - iii. Componentes personalizados integrados (autor y clase).
 - b. **Estructura del componente**
 - i. Diagrama simple o explicación de cómo se organizan los componentes.
 - ii. Layout utilizado.
 - c. **Propiedades añadidas**
 - d. **Eventos**
 - e. **Ejemplo de uso**
 - i. Código de ejemplo de cómo instanciar y configurar el componente.
 - ii. Capturas de pantalla del funcionamiento.
 - f. **Ficha por cada componente utilizado - Valoración 0 - 10 de cada ítem. Justificar cada una de las puntuaciones.**
 - i. Utilidad.
 - ii. Calidad del componente (software).
 - iii. Claridad de la documentación.
 - iv. Funcionamiento acorde con lo documentado.
2. Entregar el los ficheros .java generados.
3. Entregar el componente empaquetado en un **.jar**

Criterios de evaluación

- Correcta reutilización de componentes de la **Tarea 1**.
- Diseño limpio y coherente del componente compuesto.
- Uso adecuado de propiedades y eventos.
- Claridad y calidad de la documentación.
- Funcionamiento correcto y demostrable.
- Utilidad del componente creado.
- Valoración de las fichas.